OFFICIAL TECHNICAL STATUS REPORT

Brief! - the move from Javalin to Spring

Application 2.0.0 to 2.2.0 | Report v4 | Javalin line closed, Spring line open

Project	Brief! - a full-stack web application; a message board where every post fits in 255 characters. Domain registered: briefmessage.net.
Author	Ambroise Lumanikio (OneTechly, LLC)
Versions covered	1.1.0 (the final Javalin release), 2.0.0, 2.1.0 and 2.2.0 (Spring Boot edition)
Report	OTSR v4 | October 6, 2026 | follows OTSR v3 (October 3, 2026, application 1.0.2)
Decision recorded here	**The Javalin edition is closed at 1.1.0. OTSR v3 was its last report.** Brief! continues on the Spring framework only. The Javalin code stays in the repository, frozen, as a reference.
Overall status	**2.1.0 verified:** 133 of 133 tests pass on the real stack. **2.2.0 built and checked as far as the build environment allows; its first real run is the one open action** (Section 4).

#	Section
1	Summary
2	Decision Record: Javalin Closed, Spring Continues
3	Version History Since OTSR v3
4	Verification Record
5	Architecture of the Spring Line
6	What 2.1 and 2.2 Changed, and Why
7	Create Account Redesign
8	Defects Found and Lessons
9	Security Posture
10	Known Limitations and Open Items
11	What Comes Next
12	How to Run 2.2.0

© 2026 Ambroise Lumanikio | OneTechly

1. Summary

OTSR v3 closed with one application, one backend (Javalin with hand-written JDBC) and 67 passing tests. Three days later the project has a second backend on Spring Boot that has overtaken the first, a registered domain, and a decision about which of the two backends carries the product forward.

Item	At OTSR v3 (Oct 3)	Now (Oct 6)	Status
Javalin edition	1.0.2, 67 of 67	1.1.0, **79 of 79** on the real stack. Final release.	FROZEN
Spring Boot edition	Did not exist	2.1.0, **133 of 133** on the real stack; 2.2.0 built	ACTIVE
2.2.0 automated suite	-	149 tests written; real run not yet done	NOT RUN
Security	Random tokens kept in memory; 4-character passwords	Spring Security, signed JWT, strong passwords, login rate limit	DONE
Feed	Returns every post	Cursor paging, ten at a time	DONE
Mentions	Did not exist (added in 1.1.0 by searching every post)	Stored in an indexed table at write time (2.2.0)	DONE
Create account form	Username and password	Confirm password, show/hide, live username check	DONE
MySQL profile	-	Written; never run	UNTESTED
Deployment	Localhost and home Wi-Fi	Localhost and home Wi-Fi. Domain bought, nothing deployed.	PLANNED

The one open action

Run `.\mvnw.cmd test` in `backend-spring` of Brief_v2.2.0 and record the line `Tests run: 149, Failures: 0, Errors: 0, Skipped: 0`. Until then 2.2.0 is "built", not "verified". Every earlier release written the same way passed on its first real run, which is a good sign and not a result.

2. Decision Record: Javalin Closed, Spring Continues

Decision	Stop developing the Javalin edition. Develop Brief! on Spring Boot only.
Date	October 6, 2026
Javalin edition ends at	Version 1.1.0, reported in OTSR v3 (1.0.2) and closed here with its final test result, 79 of 79.
What happens to its code	It stays in `backend/`, unchanged. It still starts, and the web app still works with it. It receives no new features.
Why keep it at all	It is the same application written by hand. Every annotation in the Spring edition replaces code that can be read there. `docs/EDITIONS.md` maps the two file by file.

Is the reason "Spring scales better and is more reliable"?

Partly, and the difference matters when you explain the decision to someone else. A framework does not make an application scale. Design does: requests that carry their own proof of identity, queries that use an index, work done once instead of at every request. A Javalin application built that way scales, and a Spring application built carelessly does not. What Spring changes is how much of that design you have to build and maintain yourself.

© 2026 Ambroise Lumanikio | OneTechly

Claim	Accurate?	The precise version
"Spring makes the app scale."	No	Brief! scales better in 2.x because of three design changes: stateless JWT instead of an in-memory session map, paging, and the mention index. All three could have been written in Javalin.
"Spring makes it more reliable."	Partly	It supplies tested, standard parts for the things that are easy to get wrong by hand: transactions (`@Transactional`), connection pooling, the security filter chain, validation. Less custom code in those places means fewer places for defects.
"Spring makes the next steps cheaper."	Yes	Database migrations (Flyway), health and metrics endpoints (Actuator), Redis, email, scheduled jobs and payment SDKs all plug into Spring with a dependency and a few lines. In Javalin each is a small project.
"Spring is what employers ask for."	Yes	The Levrx posting names Spring, Hibernate and MySQL. The Spring edition is direct evidence for that role.
"Two editions cost double."	Yes	Every feature since 2.0 would have had to be written and tested twice. 2.1 and 2.2 already exist in one edition only.

Assessment

The decision is right. State the reasons as: one codebase instead of two, a standard and well-supported ecosystem for everything a product still needs, and the stack the job market asks for. Do not state it as "the framework scales", because an interviewer who knows the subject will push on that sentence.

What the decision costs

- **Less visibility.** Much of what runs is no longer in the project's own code. A repository method has no body; a mistake in its name appears at start-up, not at compile time.
- **A slower start and a larger jar** than the Javalin edition.
- **The build environment still cannot download Spring.** Each release is written against stand-ins and first runs for real on the author's machine. Section 4 describes how that risk is controlled.

3. Version History Since OTSR v3

Version	Date	Edition	What it added
1.1.0	Oct 4	Javalin	Clickable links in posts. Mentions: a new endpoint and a Mentions view, an unread count, on-screen and desktop alerts. The page re-reads itself every 15 seconds. **Final Javalin release.**
2.0.0	Oct 5	Spring Boot	The same API rebuilt on Spring Boot 3.3, Spring Web, Spring Data JPA (Hibernate) and Bean Validation. Entities and repository interfaces replace the DAOs. H2 by default, a MySQL profile. Deliberately no new behaviour, so the two editions could be compared.
2.1.0	Oct 5	Spring Boot	Spring Security in front of every request. Login tokens become signed JWTs. Cursor paging on the feed. Login rate limit. Strong password rule for new accounts.
2.2.0	Oct 6	Spring Boot + web app	Mentions move to their own indexed table, written in the same transaction as the post. Create account form redesigned. Javalin edition declared frozen.

© 2026 Ambroise Lumanikio | OneTechly

Measure	1.0.2 (OTSR v3)	1.1.0	2.0.0	2.1.0	2.2.0
Backend tests	67	79	82	133	149
of which unit / end-to-end	29 / 38	39 / 40	41 / 41	76 / 57	86 / 63
API operations documented	12	13	13	13	13
Database tables	2	2	2	2	3
Teaching comments in the study copy	not counted	not counted	193	250	268

4. Verification Record

All results below are from the author's Windows machine: Java 17.0.11, Maven through the Maven Wrapper.

Release	Command and date	Result	Stack	Status
Javalin 1.1.0	`.\mvnw.cmd test` Oct 6, 20:25	**Tests run: 79, Failures: 0, Errors: 0, Skipped: 0** BUILD SUCCESS, 16.6 s	Javalin 5.6.3, Jetty 11.0.17, H2	**PASS**
Spring 2.0.0	`.\mvnw.cmd test` and `package` Oct 5	**Tests run: 82, Failures: 0, Errors: 0, Skipped: 0** jar built	Spring Boot 3.3.5, Hibernate 6.5.3, Tomcat 10.1.31	**PASS**
Spring 2.1.0	`.\mvnw.cmd test` Oct 6, 02:58	**Tests run: 133, Failures: 0, Errors: 0, Skipped: 0** BUILD SUCCESS, 35.4 s	as 2.0.0, plus Spring Security 6.3.4	**PASS**
Spring 2.2.0	not yet run	Expected: Tests run: 149	as 2.1.0	**NOT RUN**

Suite by suite

Suite	Javalin 1.1.0	Spring 2.1.0	Spring 2.2.0 (expected)	What it proves
ContractTest (end to end, compat mode)	32	41	47	The original eight requirements and everything added since, over real HTTP against a real database
StrictAuthTest (end to end, strict mode)	8	16	16	Tokens, ownership, forged tokens, the password rule, the lock-out
AccountServiceTest	10	11	11	Registration and login rules; hashing; upgrade of old passwords
MessageServiceTest	8	12	15	Post rules; paging; mention rows written and removed with the post
MentionServiceTest	10	10	17	The whole-word rule; what is stored after create, edit, delete and rebuild
PasswordHasherTest	5	5	5	Hash, verify, salt uniqueness
SessionServiceTest	6	-	-	In-memory tokens (Javalin only)
JwtServiceTest	-	12	12	Sign, verify, expiry, revocation, forged and wrong-key tokens
JwtAuthenticationFilterTest	-	4	4	The filter signs a caller in, or leaves them anonymous
LoginRateLimiterTest	-	12	12	Counting, lock-out, release, 40 simultaneous attempts
AuthGuardTest, PasswordPolicyTest	-	5 + 5	5 + 5	Ownership rule; password rule
Total	**79**	**133**	**149**	

How a release is checked before its first real run

The build environment cannot download Spring, Hibernate or Mockito. Each release is therefore checked in other ways first, and the release notes say exactly what was and was not verified.

Check	What it catches	2.2.0
Compile every file against stand-ins written from the real library sources	Typing mistakes, wrong method names and signatures	49 class files, 0 errors
Run the unit tests	Wrong logic. The rules, JWT, rate limit, password and paging code are plain Java and run for real.	86 of 86
Send simulated requests through the real filter, URL rules, controllers and services, with an in-memory database that enforces the unique constraint	Wrong wiring between the layers, in both auth modes	177 checks
Break the code on purpose, one change at a time	Tests that would pass whatever the code does	8 of 8 changes caught

© 2026 Ambroise Lumanikio | OneTechly

Check	What it catches	2.2.0
Independent line-by-line review against the real Hibernate, Spring Data, Spring and Mockito sources	Anything the stand-ins get wrong about the real libraries	No blocking finding; 2 minor findings fixed
Drive the web app in a real browser against a stand-in server, for both editions	Broken screens and flows	58 checks
Compile the study copy and compare with the standard copy	A teaching comment that changed the code	Identical, byte for byte

Track record of this method: four releases written this way (1.0.2, 1.1.0, 2.0.0, 2.1.0), four first runs with zero failures. What it cannot prove, and what the 149-test run will: the new JPQL query, the creation of the `mention` table and its index, the transactions, and the 63 end-to-end tests.

Manual checks by the author, October 6

- Desktop (Edge, `localhost:5173`): signed in as Maya_zo; a post by Gilberto containing @Maya_zo shows the name highlighted, a marker on the post, the count 1 on the Mentions button and "(1) Brief!" in the tab title.
- Android phone over Wi-Fi (`192.168.1.185`): the Mentions view lists both posts that name Maya_zo, newest first; dark theme follows the phone's setting; the strong-password hint is shown under Create account.
- **One observation to follow up.** A phone screenshot taken at 12:04 shows the page itself loaded but the status line reading "Server not reachable", while one taken at 11:36 shows the app working. The likely cause is that the backend had been stopped, or the computer had gone to sleep, by 12:04. Worth confirming once: with the backend running, the phone should show "Server online".

5. Architecture of the Spring Line

The shape is the same three layers as before: controller, service, data access. What changed is who writes each part. The companion document *Brief! Backend Concepts Guide* explains every technology named here.

Layer	Package	Contents in 2.2.0	Supplied by
In front of everything	`config`, `security`	Security filter chain; JWT filter and service; ownership guard; rate limiter; password policy; auth mode	Spring Security + own code
HTTP	`controller`	Account, message and health controllers; paging helper	Spring MVC, Jackson
Input shapes	`dto`	Six records with validation rules	Bean Validation
Errors	`exception`	Five exception types and one handler for all controllers	Spring MVC
Rules	`service`	Account, message and mention services; mention backfill at start-up	own code, `@Transactional`
Data access	`repository`	Three interfaces, eleven query methods, no implementation classes	Spring Data JPA
Tables	`entity`	Account, Message, Mention	JPA / Hibernate
Database	-	H2 file by default; MySQL by profile (untested)	HikariCP, JDBC

Table	Columns	Constraints and indexes
`account`	`account_id, username, password` (a PBKDF2 hash)	Primary key; `username` unique
`message`	`message_id, posted_by, message_text, time_posted_epoch`	Primary key. No foreign key to `account` yet (Section 10).
`mention` (new in 2.2.0)	`mention_id, message_id, account_id`	Primary key; unique `(message_id, account_id)`; index `(account_id, message_id)`

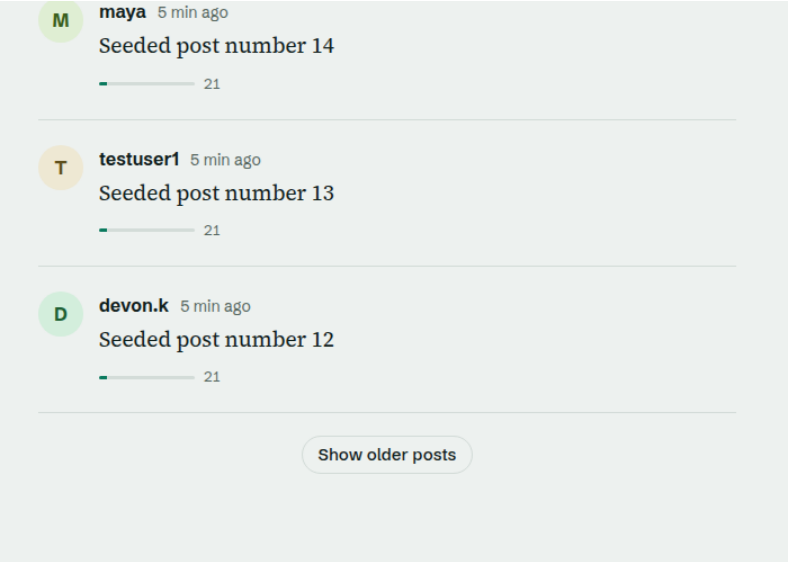
6. What 2.1 and 2.2 Changed, and Why

Change	Before	After	Why
Who checks the caller	Each controller method called a guard	Spring Security's filter chain runs first, for every request	A check that cannot be forgotten
Login token	32 random bytes, looked up in a map in memory	A JWT signed with HMAC-SHA256, checked with the key alone	No per-login memory; several servers could share the work
Sign-out	Delete the map entry	A list of revoked token ids	A signed token cannot be withdrawn any other way
Feed	Every post, every 15 seconds	Ten posts; "Show older posts" asks for the next ten below the oldest id on screen	Cost no longer grows with the age of the board
Failed logins	Unlimited	Five attempts per username and address, then 429 for 15 minutes; counted before the password is checked	Slows password guessing, including guesses sent in parallel
New passwords	4 characters	8 or more, a letter and a digit, not the username	The earlier rule was the training exercise's
Mentions (2.2)	Search every post at each request	Rows written when a post is saved; read through an index	The request is sent every 15 seconds by every signed-in browser
Writing a post (2.2)	One insert	Post and mention rows in one transaction	Both are stored, or neither

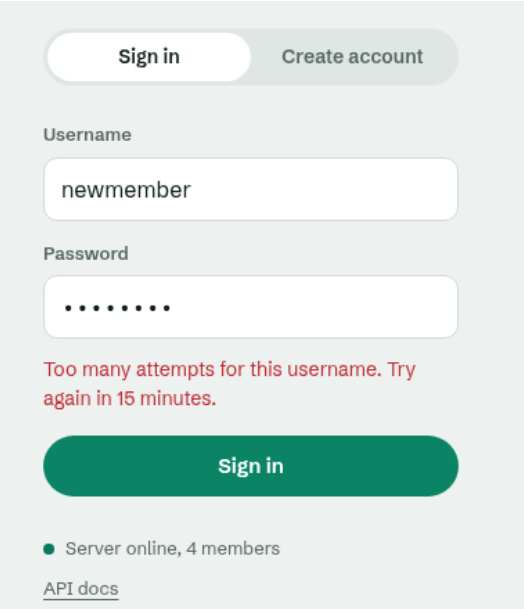
The mention table in detail

- **On create and edit**, `MentionService.index` asks the database for accounts whose username occurs in the text (one JPQL query), applies the whole-word rule to those candidates in Java, and then changes only the rows that differ from what is stored.
- **On delete**, the post's mention rows are deleted with it.
- **On the first start of 2.2 against an older database**, `MentionBackfill` indexes the existing posts once, so no one's mentions disappear with the upgrade.
- **One behaviour changed.** Who a post mentions is decided when it is written. A name typed before that member registered no longer becomes a mention later. The Javalin edition keeps the old behaviour.

© 2026 Ambroise Lumanikio | OneTechly



Paging (2.1): the feed ends with "Show older posts" when the server has more.

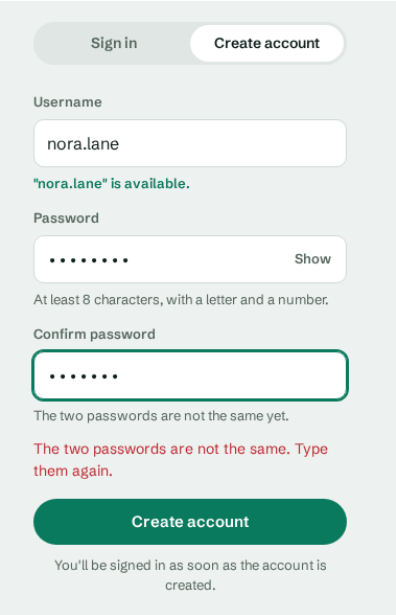


Rate limit (2.1): the sixth attempt is refused even with the right password.

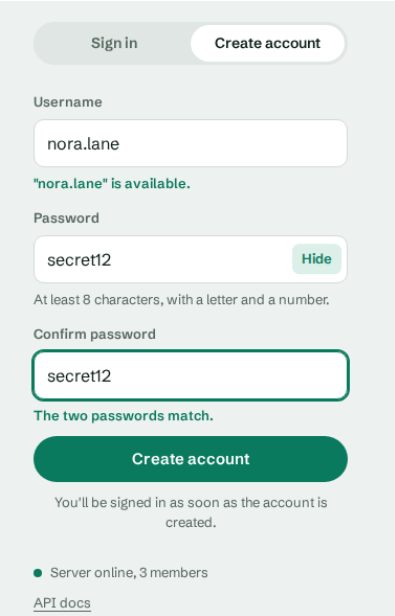
7. Create Account Redesign

Requested on October 6: keep Sign in and the password hint as they are; add a confirmation field and whatever else a registration form should have; make it work the same on desktop and phone.

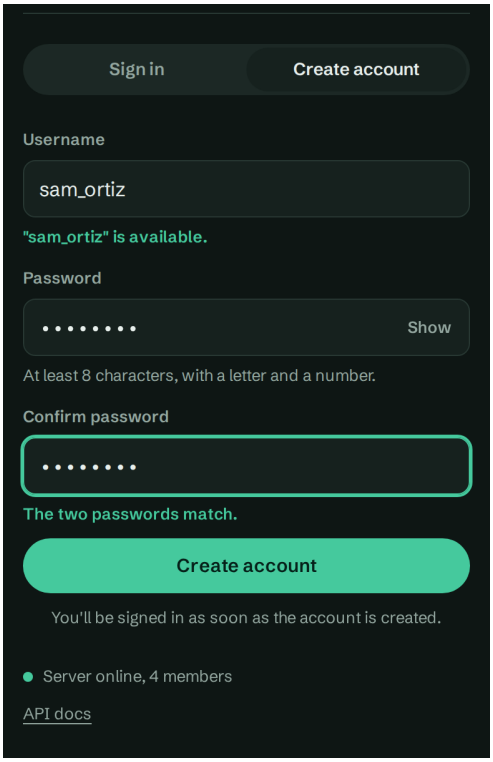
Added	Behaviour
Confirm password	A second field. The line under it reads "The two passwords are not the same yet" and then "The two passwords match" as you type.
Show / Hide	One button inside the password field reveals or masks both fields. Always masked again when you switch tabs.
Username check	The rule is shown under the field. Once the name is valid, the line reports "available" or "already taken", using the member list the page already has.
Cursor placement	On a failed check the cursor moves to the field that needs fixing. Checks run in the order of the fields.
Stale messages	An error message disappears as soon as you change the input it was about.
Unchanged	Sign in renders exactly what it did before. The "At least 8 characters, with a letter and a number" hint is untouched.



Desktop: a mismatch is reported and the cursor is in the field to fix.



Desktop: passwords shown, name available, ready to submit.



Phone, dark theme: the same form at 390 px wide.

The server did not change for this feature. "Type it twice" protects against a typing slip, which only the browser can see; the server still receives one password and still applies its own rule to it.

8. Defects Found and Lessons

None of these reached a release that was run. All were found by review or by the checks in Section 4.

#	Defect	Found by	Fix	Lesson
1	**Rate limit could be bypassed.** The count went up only after a wrong password was found, so many guesses sent at the same instant all passed the "is it locked?" check first.	Independent review of 2.1.0	Count the attempt before checking the password, in one atomic step. A 40-thread test now proves at most the limit get through.	"Check, then act" on shared data must be one step.
2	Lock-out could last one second: it was measured from the first failure, not from the one that reached the limit.	Same review	The lock runs its full time from the attempt that triggers it.	Test the boundary, not only the middle.
3	Mention search compared text lower-cased by Java with usernames lower-cased by the database. On some system languages the two disagree.	Independent review of 2.2.0	The database lower-cases both sides.	One engine should decide one question.

#	Defect	Found by	Fix	Lesson
4	Two simultaneous edits of one post could both insert the same mention row; the loser would get a 500 with a JSON body, against the empty-body contract.	Same review	The unique constraint stops the duplicate; a handler now answers 409 with an empty body.	A constraint needs an error path.
5	A failure while indexing old posts at start-up would have stopped the application from starting.	Same review	The backfill logs the error and lets the application start.	Start-up helpers must not be able to take the service down.
6	The registration form kept showing "passwords are not the same" after they had been corrected.	Screenshot review	Messages clear when the input changes; a browser check covers it.	Look at the screenshot, not only the test result.
7	Wrong jar name typed when starting 2.0.0 (`brief-api.jar`).	Author, at the terminal	README now names both jars side by side.	Two editions, two names: say so where the command is.

9. Security Posture

Area	State in 2.2.0	Status
Password storage	PBKDF2-HMAC-SHA256, 120,000 iterations, 16-byte salt per account, constant-time comparison	DONE
Password rule	8+ characters, a letter and a digit, not the username (new accounts, strict mode)	DONE
Authentication	JWT signed with HMAC-SHA256; 12-hour life; signature verified before the payload is read	DONE
Authorization	URL rules in Spring Security; ownership checked in `AuthGuard` (401 and 403 kept apart)	DONE
Guessing	Five attempts per username and address, then a 15-minute lock; parallel attempts cannot slip through	DONE
Injection	Bound parameters everywhere, including the one hand-written JPQL query	DONE
Cross-site scripting	Posts are drawn as text by React; links are limited to http and https	DONE
Signing key	From the environment (`BRIEF_JWT_SECRET`); a random key per run if not set. Never in a committed file.	DONE
Transport	Plain HTTP on localhost and home Wi-Fi. HTTPS arrives with deployment.	PLANNED
Account recovery	None: accounts have no email address, so a forgotten password cannot be reset	OPEN
Bots at registration	No CAPTCHA or email verification	OPEN
JWT and rate-limit code	Own implementations, written to be read. Tested, but not a maintained library.	ACCEPTED

10. Known Limitations and Open Items

#	Item	Effect	Priority
1	2.2.0 has not been run on the real stack	Status is "built", not "verified"	HIGH
2	MySQL profile never run	The claim "supports MySQL" is unproven	MEDIUM
3	Schema managed by `ddl-auto=update`, not by versioned migrations	No record of schema changes; unsafe for production data	HIGH
4	No foreign keys between message, mention and account	The database does not itself prevent orphan rows	MEDIUM
5	Rate-limit counts and revoked tokens live in one server's memory	Lost on restart; wrong with more than one server	MEDIUM
6	Mentions endpoint returns all of a member's mentions at once	Grows with how often a member is mentioned	MEDIUM
7	Finding mentioned members compares the post with every username	One database scan per post; fine for thousands of members	LOW
8	Usernames differing only in capitals can both register	Either spelling mentions both	MEDIUM
9	Polling every 15 seconds	Up to 15 seconds' delay; steady background load	LOW
10	Spring Security marks all responses "do not cache"	The browser re-downloads the web app's files on every visit	LOW
11	No email on accounts	No password reset, no verification, no billing contact	HIGH
12	One public board for everyone	Not yet a product a team can buy	HIGH

Items 3, 4, 5 (in part), 8 and 10 belong to the next release. Items 11 and 12 are product work and are planned in the launch roadmap.

11. What Comes Next

With a domain registered, the question is no longer only "what should the code do next" but "what has to be true before a stranger can use this". That plan is a separate document, *Brief! Launch Roadmap* (`docs/Brief_Launch_Roadmap.pdf`). In outline:

Milestone	Goal	Main contents	When
M0	The domain is safe and ready	Auto-renew, DNSSEC, an email address that forwards	NOW
M1 (2.3)	A build that can be deployed	PostgreSQL, Flyway migrations, Dockerfile, health check, foreign keys, case-insensitive usernames	NEXT
M2	briefmessage.net answers over HTTPS	Render web service and database, Cloudflare DNS	NEXT
M3	Real accounts	Email, verification, password reset, terms and privacy pages	LATER
M4	Private boards for teams	Workspaces, invitations, roles	LATER
M5	First payment	Stripe Checkout, customer portal, webhooks, plan limits	LATER
M6	Launch	Monitoring, backups, support address, landing page	LATER

© 2026 Ambroise Lumanikio | OneTechly

> **Order of priorities**
>
> Round 2 of the Levrx interview is a take-home task due this week. It comes before every milestone above. Nothing on this page is urgent; the domain is registered and loses nothing by waiting.

12. How to Run 2.2.0

```
cd Brief_v2.2.0\brief\backend-spring
.\mvnw.cmd test                     # expect: Tests run: 149, Failures: 0, Errors: 0, Skipped: 0
.\mvnw.cmd package                  # builds target\brief-spring.jar
java -jar target\brief-spring.jar   # wait for "Started BriefApplication"; open http://localhost:8080
```

- A new folder has its own empty database (`backend-spring\h2`). Register the test members again; passwords need 8 or more characters with a letter and a number.
- To keep the members and posts from an earlier folder instead, copy its `backend-spring\h2` folder into the new one before starting. The mention table is then filled from the existing posts at the first start; the log says how many.
- On a phone on the same Wi-Fi: `http://<the computer's IPv4 address>:8080`. The backend must be running and the computer awake.
- The Javalin edition, for comparison only: `cd backend`, then `java -jar target\brief-api.jar` after `.\mvnw.cmd package`. Stop one backend before starting the other; both use port 8080.